

Student 6 **DAYYAM JEEVANA REDDY** 192411259RD

5 / 9 submitted

Q1. Banking Transaction System – Exception Handling

CODE

```
class Main
{
    public static void main(String[] args)
    {
        double balance = 10000, amount=3000;
        try
        {
            if(amount<=0)throw new IllegalArgumentException("Invalid amount");
            if(amount>balance)throw new InsufficientBalanceException("Insufficient Balance");
            System.out.println("Withdrawal successful");
        }
        catch(Exception e)
        {
            System.out.println("Transaction completed");
        }
    }
}
class InsufficientBalanceException extends Exception
{
    InsufficientBalanceException(String s)
    {
        super(s);
    }
}
```

INPUT

```
10000 3000
10000 12000
10000 500
abc 3000
```

OUTPUT

```
Withdrawal successful
```

Q2. Hospital Patient Registration – User-Defined Exception

CODE

```
class Main
{
    public static void main(String[] args)
    {
        int age=35;
        double temp=98.6;
        try
        {
            if(age<18)throw new InvalidAgeException();
            if(temp<98||temp>100)throw new InvalidTemperatureException();
            System.out.println("Patient registered successfully");
        }
        catch(Exception e)
        {
            System.out.println(e.getMessage());
        }
        finally
        {
            System.out.println("Registration completed");
        }
    }
}

class InvalidAgeException extends Exception
{
    InvalidAgeException()
    {
        super("InvalidAgeException");
    }
}

class InvalidTemperatureException extends Exception
{
    InvalidTemperatureException()
    {
        super("InvalidTemperatureException");
    }
}
```

INPUT

Arun,35,98.6
Heena,12,98.4

OUTPUT

Patient registered successfully
Registration completed

Q3. E-Commerce Order Processing – Built-in Exceptions

CODE

```
class Main
{
    public static void main(String[] args)
    {
        int index = 1, qty = 2;
        double [] price = {100, 200, 300};
        try
        {
            if(qty <= 0)
            {
                throw new IllegalArgumentException("Invalid quality");
            }
            System.out.println("Total : "+price[index]*qty);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Invalid product index");
        }
        catch(IllegalArgumentException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

INPUT

```
1,2
5,2
2,-1
abc,2
```

OUTPUT

```
Total : 400.0
```

Q4. Railway Ticket Booking – Multithreading and Synchronization

— Not Submitted —

Q5. Bank Account – Concurrent Withdrawals

— Not Submitted —

Q6. Online Food Delivery – Multiple Threads and Execution Order

CODE

```
class Main{
    static void order()
    {
        System.out.println("Order Processing");
    }
    static void payment()
    {
        System.out.println("Payment processing");
    }
    static void delivery()
    {
        System.out.println("Delivery Assignment");
    }
    public static void main(String[] args)throws Exception
    {
        Thread t1 = new Thread(()->order());
        Thread t2 = new Thread(()->payment());
        Thread t3 = new Thread(()->delivery());
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
    }
}
```

INPUT

Order=Pizza;Payment=UPI;Delivery=Agent A
Order=Burger;Payment=Card;Delivery=Agent B
Payment=Failed

OUTPUT

Order Processing
Payment processing
Delivery Assignment

Q7. Hospital Emergency System – Thread Priority

CODE

```
class Main
{
    static String b=null;
    static synchronized void produce(String s)throws Exception
    {
        while(b!=null)
        {
            Main.class.wait();
        }
        b=s;
        System.out.println("Produced"+s);
        Main.class.notify();
    }
    static synchronized void consume()throws Exception
    {
        while(b==null)
        {
            Main.class.wait();
        }
        System.out.println("Consumed"+b);
        b=null;
        Main.class.notify();
    }
    public static void main(String[] args)throws Exception
    {
        Thread t1=new Thread(()->
        {
            try
            {
                produce("P1");produce("P2");produce("P3");
            }
            catch(Exception e)
            {}
        });
        Thread t2=new Thread(()->
        {
            try
            {
                consume();consume();consume();
            }
            catch(Exception e)
            {}
        });
        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }
}
```

INPUT Emergency,general,Billing

OUTPUT ProducedP1
ConsumedP1
ProducedP2
ConsumedP2
ProducedP3
ConsumedP3